

2020011089_徐潇悦_光线追踪大作业

目录：

Part0 : 得分点

Part1 : 算法选型

Path Tracing 路径追踪

SPPM : Stochastic Progressive Photon Mapping 随机渐进光子映射

Part2 : 功能

参数曲面求交

重心插值

纹理映射

景深

软阴影

焦散

抗锯齿

Part3 : 加速

KDTree加速

包围盒加速

OpenMP设备加速

Part4 : 效果图

部分参考资料

Part0 : 得分点

1. 算法选型

- 实现了 **Path Tracing** 路径追踪
- 实现了 **SPPM** 随机渐进光子映射

2. 功能

- 参数曲面求交
- 三角面片重心插值
- 纹理映射
- 景深
- 软阴影
- 抗锯齿
- 焦散

3. 加速

- **KDTree** 加速观察点查询

- 包围盒 加速
- OpenMP 设备加速

Part1 : 算法选型

我实现了Path Tracing和SPPM两种方法，具体原理和代码如下：

Path Tracing 路径追踪

1. 原理

蒙特卡洛路径追踪：

为了得到相机观察的结果，要知道每一个像素对应的观察点，以及观察点上出射光线(方向朝向相机)的情况。记某点上单位立体角、单位垂直面积的辐射功率为 $L(p, w)$ ，入射光线和出射光线分别记为 $L_i(p, w)$ 和 $L_o(p, w)$ ，BRDF函数记为 $f_r(w_i \rightarrow w_o)$ ，则有公式：

$$L_o(p, w_o) = \int f_r(w_i \rightarrow w_o) L_i(p, w_i) \cos \theta_i dw_i$$

由于 $L_i(p, w)$ 的计算涉及到多次迭代，这一积分并不好求。采用蒙特卡洛积分来计算，即对函数值进行多次采样，以均值作为积分值的近似：

$$L_o(p, w) = \frac{1}{N} \sum_{i=1}^N \frac{L_i(p, w_i) f_r(p, w_i, w_o) n \cdot w_i}{p(w_i)}$$

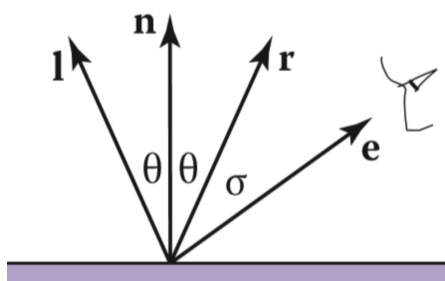
具体到操作上来说，可以对每个像素点的观察视线进行随机采样，计算场景中沿此观察实现方向的光线强度，对采样值取平均作为像素点的值。

光线追踪计算：

光线在物体表面有三种情况：漫反射、镜面反射、折射

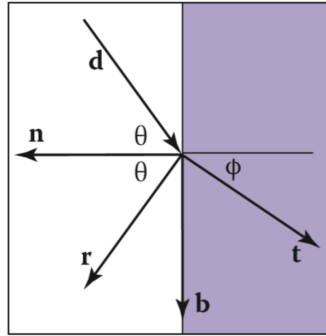
漫反射：在半圆面内随机选择一个方向做镜面反射

镜面反射： $\vec{r} = 2 * \vec{n} * (\vec{l} \cdot \vec{n}) - \vec{l}$



折射： $\cos^2 \phi = 1 - \frac{n_t^2}{n_i^2} (1 - \cos^2 \theta)$ $\vec{t} = \sin \phi * \vec{b} - \cos \phi * \vec{n}$ $\vec{b} = \frac{\vec{d} + \vec{n} * \cos \theta}{\sin \theta}$

$$\vec{t} = \frac{n * (\vec{d} - \vec{n} * (\vec{d} \cdot \vec{n}))}{n_t} - \vec{n} \sqrt{1 - \frac{n^2 (1 - (\vec{d} \cdot \vec{n})^2)}{n_t^2}}$$



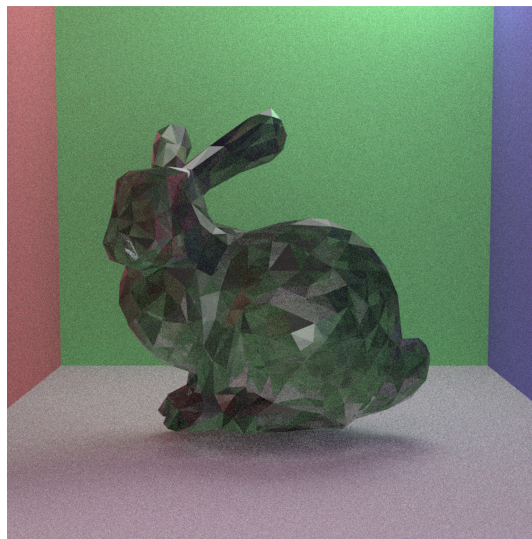
2. 代码：raytrace.h

- a. void Operate(Image &image): 遍历image上的所有像素点，引入subpixel来抗锯齿
- b. Vector3f radiance(.....): 光线迭代传播

代码全部位于<raytrace.h>中。由于行数较多，不在报告里面贴出。

3. 效果：

较暗光线下的兔子：



<pathtrace_bunny.bpm>

SPPM : Stochastic Progressive Photon Mapping 随机渐进光子映射

1. 原理

SPPM分为两个阶段：

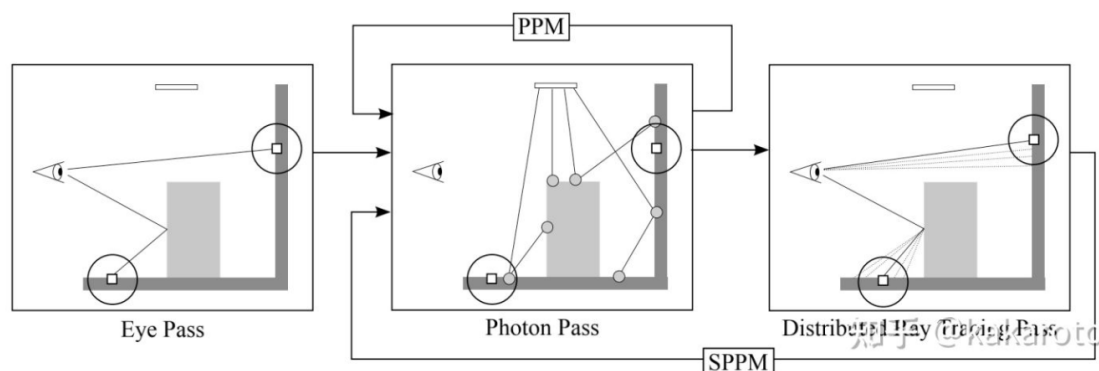
Part1 : RayTracing

从相机进行路径追踪，在追踪到漫反射表面的时候记录下该观察点，存储在合适的数据结构中。

Part2 : PhotonTracing

从光源发射光子，到达漫反射表面时将光子信息收纳进附近的观察点内。每个观察点统计附近的光子信息，并根据这些信息计算出最终的辐射率（radiance）。

SPPM每次发射一些随机分布的观察视线，然后直接用观察视线的值更新像素。能起到比PPM降低存储开销的效果。



下面是两个Part的具体内容：

<Preprocess> 初始化参数，建立空的ObservePoints表，获取illuminants信息。

<RayTracing Pass>

按每个像素点进行采样，随机sample一个方向作为观察射线方向。在ObservePoints表中(x,y)对应的位置记录下观察点。因为直接射入眼睛的光线不会经过漫反射，所以对于光源直射的部分，用一个fluxLight来记录，每次经过光源，就补充fluxLight值。Ray在传播的情况中，每经过一次折射或者反射，会面临损耗，用一个Vector3f decrease来记录损耗值，具体实现上，认为颜色越黑的表面引起越多的损耗。光线的方向变化同PathTracing的操作。

<Photon Pass>

经过上一个Pass，已经建立好观察点图。现在对于每一个光源，sample出一些光子，光子在场景中传播，用光源的emission生成光子的能量，光子能量在传播过程中不断衰减。光子的方向变化同PathTracing的操作。如果光子打在漫反射表面上，就记录光子。

一些细节操作：

<KDTree>

为了加快每次查找观察点的速度，引入了KDTree。基本思想是根据x\y\z维依次进行划分。

<能量和像素值计算>

i 表示photon pass的次数， $\tau_i(S, w)$ 表示像素S内的累积能量， $R_i(S)$ 表示收集半径。

$M_i(x_i)$ 表示观察点 x_i 接受的光子数， $\Phi_i(x_i, w)$ 表示该观察点的辐射通量。

$$N_{i+1}(S) = N_i(S) + \alpha M_i(x_i)$$

$$R_{i+1}(S) = R_i(S) \sqrt{\frac{N_i(S) + \alpha M_i(x_i)}{N_i(S) + M_i(x_i)}}$$

$$\Phi_i(x_i, w) = \sum_{p=1}^{M_i(x_i)} f_r(x_i, w, w_p) \Phi_p(x_p, w_p)$$

$$\tau_{i+1}(S, w) = (\tau_i(S, w) + \Phi_i(x_i, w)) \frac{R_{i+1}(S)^2}{R_i(S)^2}$$

对于像素S的辐射度贡献均值：

$$L(S, w) = \lim_{i \rightarrow \infty} \frac{\tau_i(S, w)}{N_e(i) \pi R_i(S)^2}$$

2. 代码

主要包含 **SPPM** 类，定义了三个主要函数：

void Operate(Image& image) : 执行整个渲染过程。

void RayTrace_GenerateObservePoints : 执行RayTrace Pass，形成观察点。

void PhotonTrace : 执行PhotonTrace Pass。

SPPM类代码全部位于<ppm.h>中。因为行数较多，不在此贴出。

光子更新代码位于<KD_Tree.hpp>中。

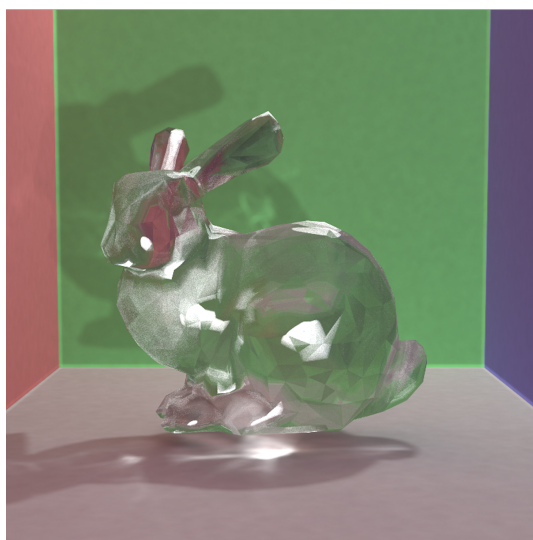
```

...
/* 如果找到自己位于其范围内的观察点 */
if ((photon->node->hit->position).squaredLength() <= node->hit->r2) {
    Hit *hp = node->hit;
    /* 按 R_{i+1} = R_i * \sqrt{(N_i + \alpha * M_i) / (N_i + M_i)} 来更新R值 */
    /* r2是r^2所以不用根号计算；M_i = 1 */
    double dec_factor = (hp->n * ALPHA + ALPHA) / (hp->n * ALPHA + 1.);
    Vector3f dr = d - hp->normal * (2 * Vector3f::dot(d, hp->normal));
    hp->n++;
    hp->r2 = hp->r2 * dec_factor;
    /* tau_{i+1} = (tau_i + Phi_i) * (R_{i+1} / R_i)^2 */
    hp->flux = (hp->flux + hp->decrease * decrease) * dec_factor;
}

```

3. 效果

用sppm渲染的兔子，有明显的焦散效果：



<spmm_bunny.bmp>

Part2 : 功能

参数曲面求交

参考了ppt给出的解法：

记参数曲面为 $P(u, v)$ ，射线为 $L(t)$ 。则交点处满足 $L(t) - P(u, v) = 0$ 。

低次曲面求交可以直接求解方程： $F(x) = F(t, u, v) = L(t) - P(u, v) = 0$

高次曲面求可以用牛顿法迭代： $x_{i+1} = x_i - \frac{F(x_i)}{F'(x_i)}$

迭代的初值可以用光线和包围盒的交点，以步长过小或者超过最大迭代次数作为终止条件。

(t_i, u_i, v_i) 处的法向： $\vec{n} = (\frac{\delta P}{\delta u} \times \frac{\delta P}{\delta v})|_{(t_i, u_i, v_i)}$

具体来说，可以用 μ 和 θ 来表示参数表面上的点， μ 是参数曲线的自变量， θ 表示转角。用 t 表示曲面和光线的交点 $p = t \cdot d + o$ ，求解 $|p - f(\mu, \theta)|^2$ 的最小值来判断求交结果和交点的位置。

代码：<revsurfave.hpp>

求交代码都位于<revsurfave.hpp>中，主要涉及 **intersect** 和 **NewtonMethod** 两个函数。

```

bool intersect(const Ray &ray, Hit &h, double tmin) override {
    double t, theta, mu;
    /* 包围盒加速 */
    if(!AABB.intersect(ray, t) || t > h.getT()) return false;
    /* 以包围盒的交点作为迭代初值 */
    Vector3f position = ray.origin + ray.direction * t;
    theta = atan2(-position.z(), position.x()) + M_PI;
    mu = (pCurve->ymin - position.y()) / (pCurve->ymin - pCurve->ymin);
    Vector3f normal, point;
    /* 牛顿法求交无解 */
    if(!NewtonMethod(ray, t, theta, mu, normal, point)) return false;
    if(!isnormal(mu) || !isnormal(theta) || !isnormal(t)) return false;
    /* 不符合交点条件 */
    if(t < 0 || mu < pCurve->range[0] || mu > pCurve->range[1] || t > h.getT()) return false;
    normal = normal.normalized();
    h.set(t, material, normal, point, material->getColor(theta / (2 * M_PI), mu));
    return true;
}

```

效果：

用PathTrace渲染的参数曲面花瓶：



<pathtrace_vase.bmp>

重心插值

重心插值可以用比较少的三角形面片实现比较平滑的物体效果。

设三角形三个点的坐标分别为A、B、C，其内一点坐标为P，可以得到分割的三个三角形面片的面积：

$$S_1 = \frac{|(B-P) \times (C-P)|}{2} \quad S_2 = \frac{|(C-P) \times (A-P)|}{2} \quad S_3 = \frac{|(A-P) \times (B-P)|}{2}$$

三个点对应的法向量为 n_A, n_B, n_C , P点的法向量为：

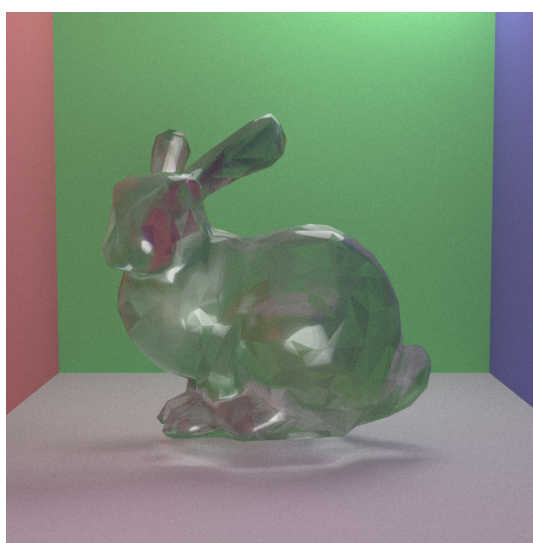
$$n_p = \frac{S_1 * n_A + S_2 * n_B + S_3 * n_C}{S_1 + S_2 + S_3}$$

代码：<triangle.hpp>

```
bool intersect(const Ray& r, Hit& h, double tmin) override {
    //
    Vector3f origin = r.getOrigin();
    Vector3f direct = r.getDirection();
    //
    /* 利用Cramer法则判断射线和三角形是否相交 */
    Vector3f E1 = A - B;
    Vector3f E2 = A - C;
    Vector3f S = A - r.getOrigin();
    double t = Matrix3f(S, E1, E2).determinant() / Matrix3f(r.getDirection(), E1, E2).determinant();
    double beta = Matrix3f(r.getDirection(), S, E2).determinant() / Matrix3f(r.getDirection(), E1, E2).determinant();
    double gamma = Matrix3f(r.getDirection(), E1, S).determinant() / Matrix3f(r.getDirection(), E1, E2).determinant();
    if (beta < 0 || beta > 1 || gamma < 0 || gamma > 1 || beta + gamma > 1 || t < tmin || t > h.getT())
        return false;
    //
    /* 三角插值计算 */
    Vector3f point(origin + direct * t);
    Vector3f va = (vertices[0] - point), vb = (vertices[1] - point), vc = (vertices[2] - point);
    double ra = Vector3f::cross(vb, vc).length();
    double rb = Vector3f::cross(vc, va).length();
    double rc = Vector3f::cross(va, vb).length();
    //
    Vector3f N = normal;
    if (!NSet) N = (ra * an + rb * bn + rc * cn).normalized();
    //
    if (Vector3f::dot(normal, r.getDirection()) < 0)
        h.set(t, this->material, N, point, material->diffuseColor);
    else
        h.set(t, this->material, -N, point, material->diffuseColor);
    return true;
}
//
```

效果：

用PathTrace渲染出来的兔子，同样是bunny_1k.obj 模型，可以看出兔子表面明显变得更加平滑。



<pathtrace_bunny_2.bmp>

纹理映射

纹理映射需要做从三位坐标到二维左边的转化。具体来说：

<球体>

参考了图形学教材中的实现:

已知球面的参数方程为

$$x = \cos(2\pi u)\cos(2\pi v)$$

$$y = \sin(2\pi u)\cos(2\pi v)$$

$$z = \sin(2\pi v)$$

$$\text{则有}(u, v) = \left(\frac{2\arctan2(y, x)}{2\pi}, \frac{\arcsin(z)}{2\pi} \right)$$

<参数曲面>

直接利用求解得到的 θ 和 μ 可知 $(u, v) = \left(\frac{\theta}{2\pi}, \mu \right)$

代码：

<sphere.hpp>

```
bool intersect(const Ray &r, Hit &h, double tmin) override {  
    // 代数方法进行射线和球面求交  
    Vector3f o(r.getOrigin()), d(r.getDirection());  
    Vector3f OC = center - o;  
    double b = Rd * (Ro - Pc);  
    double c = -Vector3f::dot(OC, d);  
    double c = (Ro - Pc)^2 - r^2;  
    double c = OC.squaredLength() - radius * radius;  
    double delta = b * b - c;  
    if (delta <= 0) return false;  
    double sqrt_delta = sqrt(delta);  
    double t1 = (-b - sqrt_delta), t2 = (-b + sqrt_delta);  
    double t;  
    if (t1 <= h.getT() && t1 >= 0)  
        t = t1;  
    else if (t2 <= h.getT() && t2 >= 0)  
        t = t2;  
    else  
        return false;  
    Vector3f OP = o + d * t - center;  
    Vector3f normal = OP.normalized();  
    // 带入公式得到u和v  
    double u = 0.5 + atan2(normal.x(), normal.z()) / (2 * M_PI);  
    double v = 0.5 - asin(normal.y()) / M_PI;  
    h.set(t, material, getNormal(normal, OP, u, v),  
        o + d * t, material->getColor(u, v));  
    return true;  
}
```

效果：

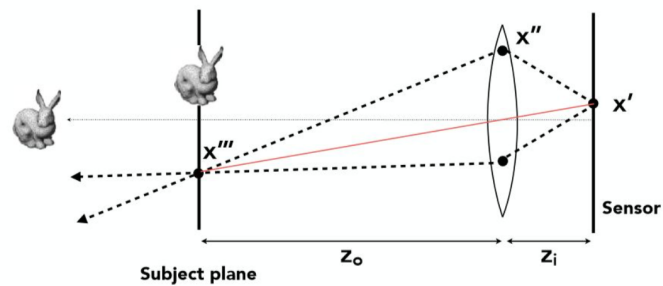
地球纹理贴图，使用sppm渲染。



<spm_earth.bmp>

景深

景深的实现方法是把模拟小孔成像方式的相机转换成模拟透镜方式成像的相机。Games101提供了一种实现景深的办法：对每个像素点，在透镜平面上随机sample一个点，计算射线方向。



代码：<camera.hpp>

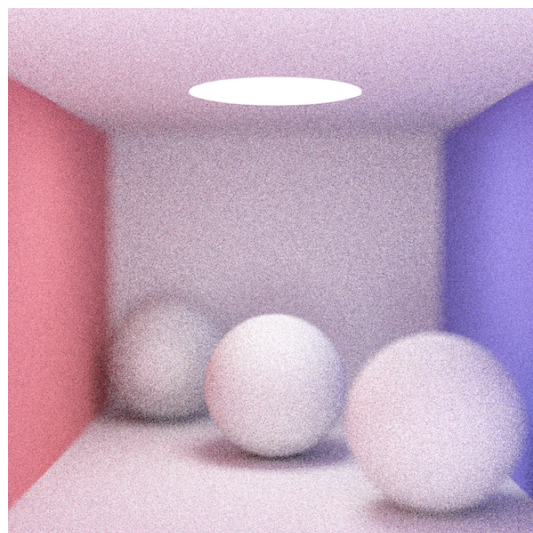

```

Ray generateRay(const Vector2f &point) override {
    ..... /* 点到透镜的距离 */ .....
    ..... float image_to_lens_x = focal_length * (point[0] - cx) / fx; .....
    ..... float image_to_lens_y = focal_length * (point[1] - cy) / fy; .....
    ..... /* 在透镜平面上sample出一个点 */ .....
    ..... float lens_x = RND1 * aperture; .....
    ..... float lens_y = RND1 * aperture; .....
    ..... /* 射线方向 */ .....
    ..... Vector3f dir(image_to_lens_x - lens_x, -image_to_lens_y - lens_y, focal_length); .....
    ..... /* 和之前相同的方法生成方向 */ .....
    ..... Matrix3f matrix(horizontal, -up, direction); .....
    ..... dir = (matrix * dir).normalized(); .....
    ..... .....
    ..... Ray ray(center + horizontal * lens_x - up * lens_y, dir); .....
    ..... return ray; .....
    ..... .....
}

```

效果：

用PathTrace渲染的图片，可以看到只有中间的球是清晰的，景深效果还是很明显的。



<pathtrace_balls.bmp>

软阴影

SPPM自带软阴影效果，效果图参考SPPM算法部分。

焦散

SPPM自带焦散效果，效果图参考SPPM算法部分。

抗锯齿

<Path Trace> : 对每个像素点依次取(0,0)(0,+1)(+1,0)(+1,+1)的subpixel, 同时加上[-1,1]的随机扰动, 四个subpixel平均后的值作为该像素点的值。这样可以使像素之间的差值变得柔和, 起到抗锯齿的作用。

<SPPM> : 由于需要建立和像素对应的观察点图, SPPM主要对每个像素取[-1,1]的随机扰动以抗锯齿。

Part3 : 加速

KDTree加速

在SPPM中使用了KDTree加速。

<建树过程>

从根节点开始递归。按照x/y/z的顺序切换分割的维度, 对所有节点进行排序, 取中间值作为本节点存储的观察点。位于本节点左侧的全部纳入左子树, 右侧的全部纳入右子树。递归建树直到所有节点纳入树中。

代码 :

```
...
// * === build():根据目前的ObservePoints坐标递归建树 === */
KDTreeNode * build(int l, int r, int dimension){
    // * 新建一个节点 *
    Vector3f INF = Vector3f(INT_MAX, INT_MAX, INT_MAX);
    KDTreeNode * p = new KDTreeNode(INF, -INF, 0);
    // * 找出[l, r]范围内xyz坐标的最大值、最小值 *
    for(int i = l; i <= r; i++){
        p->min = ZextuTaiMin(p->min, ObservePoints[i]->position);
        p->max = ZextuTaiMax(p->max, ObservePoints[i]->position);
        p->maxr = max(p->maxr, ObservePoints[i]->r2);
    }

    int m = l + r >> 1;
    // * 根据dimension来进行排序 *
    if(dimension == 0){
        nth_element(ObservePoints + l, ObservePoints + m, ObservePoints + r + 1, compareX);
    } else if(dimension == 1){
        nth_element(ObservePoints + l, ObservePoints + m, ObservePoints + r + 1, compareY);
    } else{
        nth_element(ObservePoints + l, ObservePoints + m, ObservePoints + r + 1, compareZ);
    }
    // * 新建节点选择排序得到的中点 *
    p->hit = ObservePoints[m];
    // * 递归建树, 知道所有节点纳入树中 *
    if(l <= m - 1){
        p->lc = build(l, m - 1, (dimension + 1) % 3);
    } else{
        p->lc = nullptr;
    }
    if(m + 1 <= r){
        p->rc = build(m + 1, r, (dimension + 1) % 3);
    } else{
        p->rc = nullptr;
    }
    return p;
}
```

包围盒加速

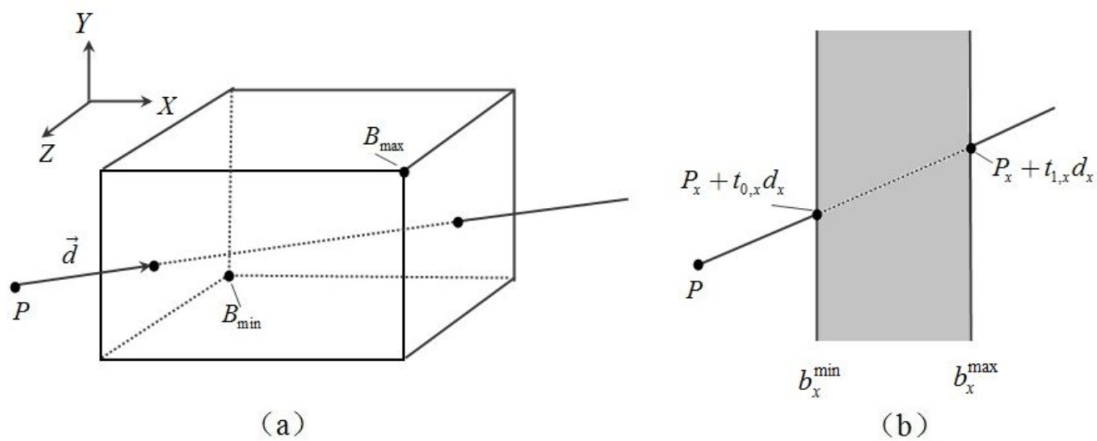
使用了AABB包围盒加速, 主要用在mesh和参数曲面的求交上。

<光线和包围盒求交>

若发生光线和平面之间的区域交叠(光线进入平面出的最大t值小于光线离开平面处的最小t值)，光线和平面就会相交。

t值可以用 $t_i = \frac{(d_{plane_i} - origin_i)}{direction_i}$ 来计算：

$$P_x + t_{0,x}d_x = b_x^{min} \quad P_x + t_{1,x}d_x = b_x^{max} \quad \rightarrow \quad t_{0,x} = \frac{b_x^{min} - P_x}{d_x} \quad t_{1,x} = \frac{b_x^{max} - P_x}{d_x}$$



代码：<bbox.h>

```

/* 计算光线是否和包围盒相交 */
bool intersect(const Ray ray, double& t_min){
    Vector3f origin = ray.getOrigin();
    Vector3f direction = ray.getDirection();
    Vector3f inv_direction = 1 / direction;
    Vector3f signals = {static_cast<float>(direction.x() < 0), static_cast<float>(direction.y() < 0),
        static_cast<float>(direction.z() < 0)};
    t_min = INT_MAX;

    double tmin, tmax, tymin, tymax, tzmin, tzmax;
    /* 首先处理yz平面,判断射线和两个yz平面的相交情况 */
    /* 获取和包围盒各平面交点的tmin和tmax值 */
    tmin = (get(signals[0]).x() - origin.x()) * inv_direction.x();
    tmax = (get(1-signals[0]).x() - origin.x()) * inv_direction.x();
    tymin = (get(signals[1]).y() - origin.y()) * inv_direction.y();
    tymax = (get(1-signals[1]).y() - origin.y()) * inv_direction.y();
    if(tmin > tymax || tmax < tymin) return false;
    /* 更新tmin和tmax */
    tmin = tmin > tymin ? tmin : tymin;
    tmax = tmax < tymax ? tmax : tymax;
    tzmin = (get(signals[2]).z() - origin.z()) * inv_direction.z();
    tzmax = (get(1-signals[2]).z() - origin.z()) * inv_direction.z();
    if(tmin > tzmax || tmax < tzmin) return false;
    /* 更新tmin和tmax */
    tmin = tmin > tzmin ? tmin : tzmin;
    tmax = tmax < tzmax ? tmax : tzmax;
    t_min = tmin;
    return true;
}

```

OpenMP设备加速

因为用自己的电脑渲染过于缓慢，我短暂地租了一台64核服务器，用OpenMP实现了多线程加速。

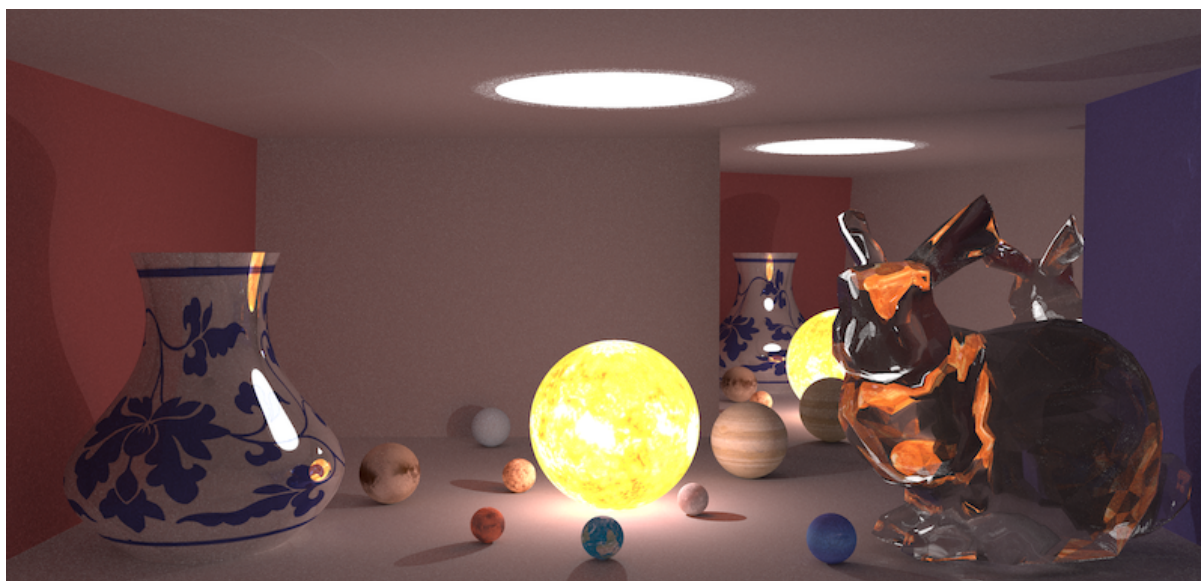
<Path Trace>：对每一行像素可以并行计算。

<SPPM>：RayTrace和PhotonTrace阶段都可以对行并行计算

提速还是很明显的，极大提高了渲染的舒适度。

Part4：效果图

用sppm渲染的场景。这里放的图是缩小过的。原图是3200*1536大小，在result文件夹中。



<sppm_final.bpm>

部分参考资料

<http://www.kevinbeason.com/smallpt/> : smallpt

<https://drive.google.com/file/d/0B8g97JkuSSBwUENiWTJXeGtOHFmSm51UC01YWtCZw/view?resourcekey=0-BSMES1GmPEtIIcv6EgBTjw> : smallpt slide

<https://www.bilibili.com/video/av90798049/> : Games101

<https://zhuanlan.zhihu.com/p/146714484> : Path Tracing

<https://zhuanlan.zhihu.com/p/385099523> : SPPM

<https://zhuanlan.zhihu.com/p/138259656> : AABB包围盒求交

<https://www.openmp.org> : OpenMP

<https://blog.csdn.net/a380880304/article/details/69787424> : 雾效

<https://blog.csdn.net/oqgeric61495217/article/details/118678509> : 纹理素材